

ECE532 Final Demo: Real-time 2D Motion Capture System

Group #9

Stephen Lee

Ethan Luan

Tiger Luo

Cary Wu



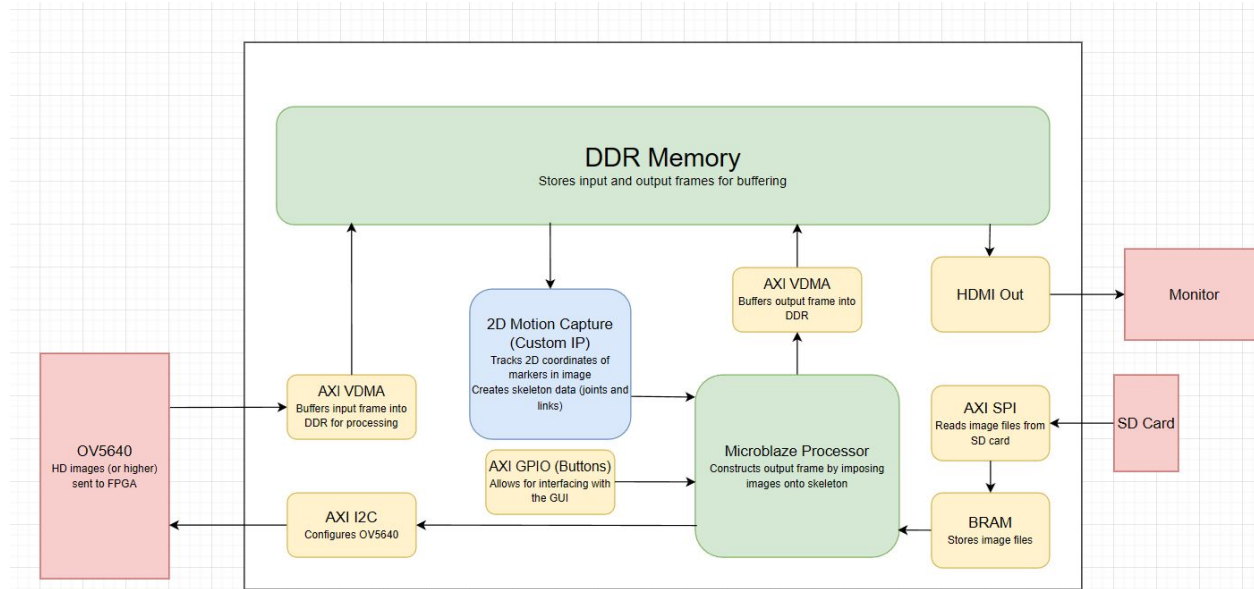
Project Overview

- Motion capture is the process of digitally tracking movement patterns.
- Variety of applications including animation, film, video games, and sports.
- Variety of methods including sensors, cameras, and markers.



Project Goal and Original Block Diagram

“Develop a real-time 2D motion capture system that captures video and processes the footage to classify markers and locate joints, with the processed video stream being output to an external monitor via HDMI.”



IP Used:

Digilent:

- DVI2RGB - Converts TMDS clock and data into RGB888 video output and timing output
- RGB2DVI - Convert timing and RGB888 video input to TMDS output
- Axi_dynclk - Dynamic clock generator for making PXLCLK for TMDS

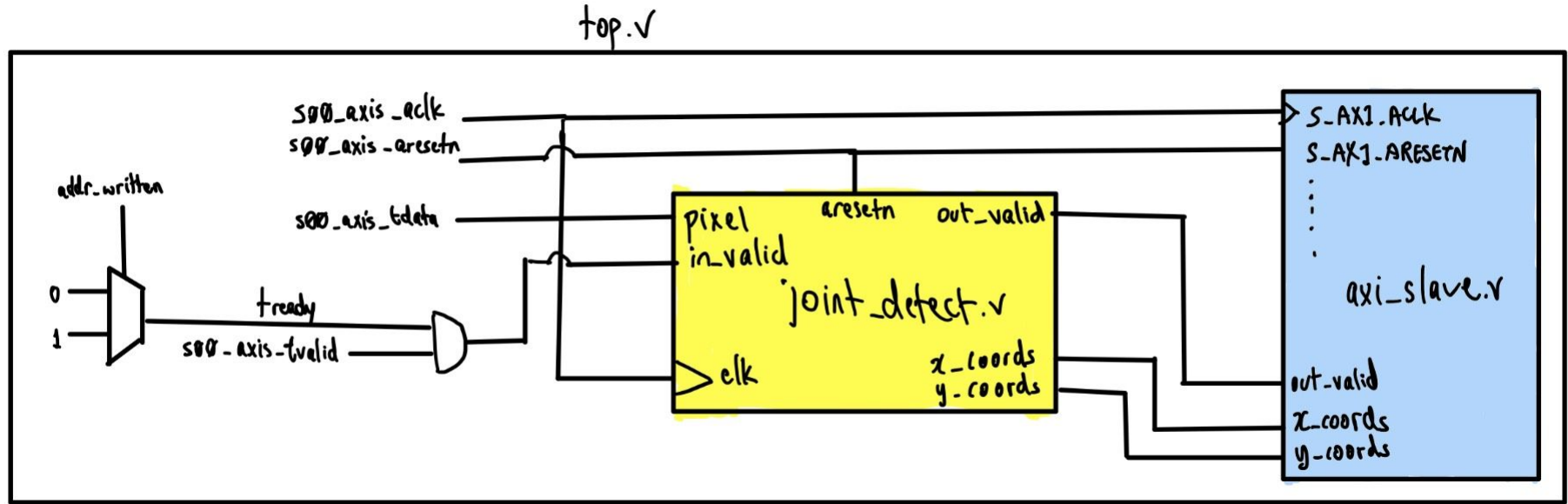
Custom IP:

- Marker detector - Has an AXIS slave and AXI-lite slave interface, and interrupt output

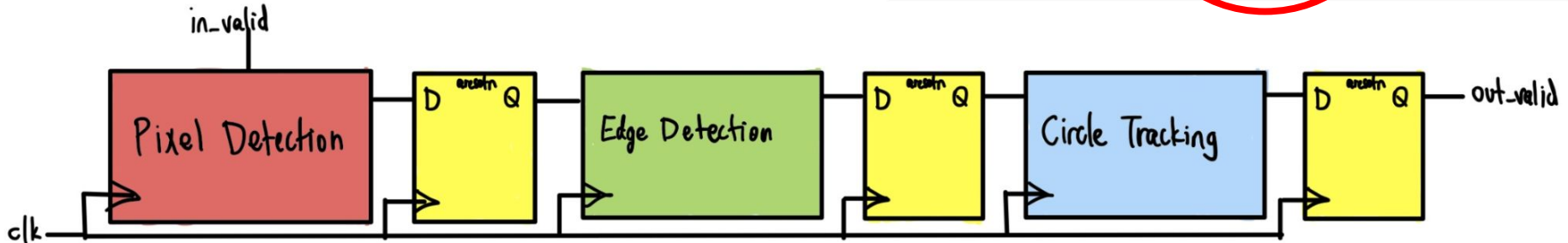
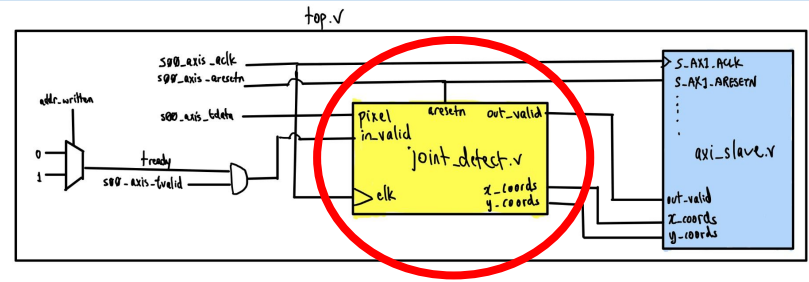
Xilinx/AMD IP:

- VDMA
- Microblaze
- VTC
- Video in/out to AXIS
- MIG7
- AXI Uartlite and GPIO

Custom IP



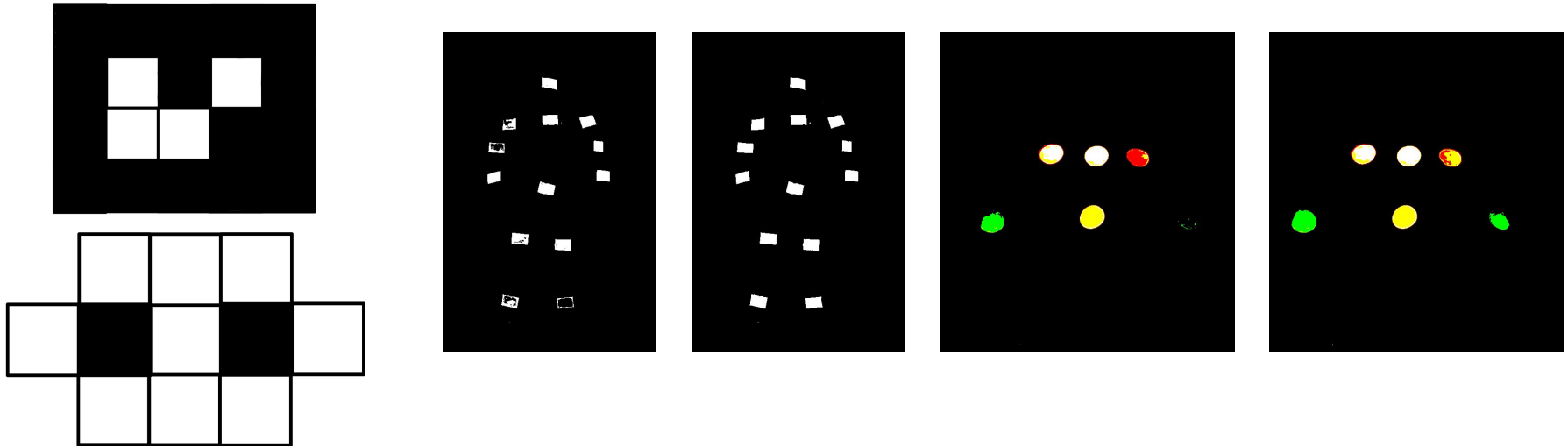
Custom IP - Joint Detection



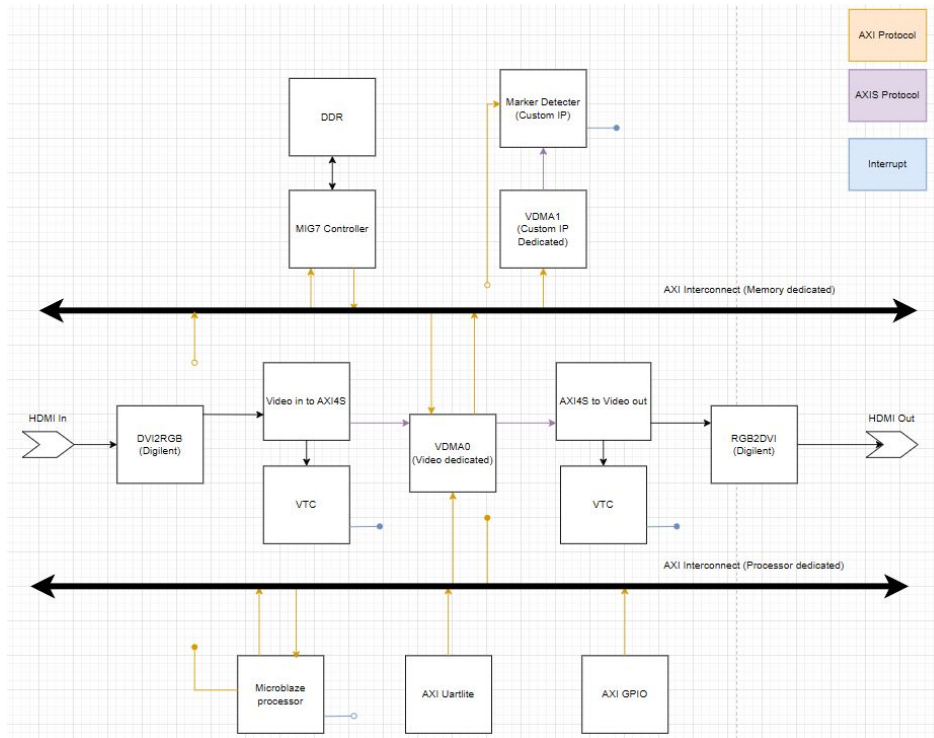
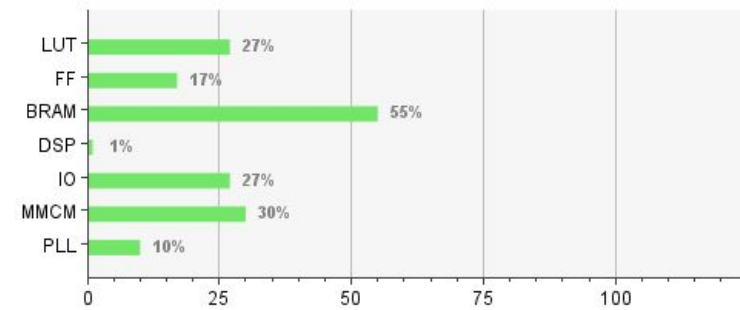
- Inputs: Stream of RGB888 pixels
- Pixel Detection:
 - Valid, Colour, Location
- Edge Detection:
 - Rising/Falling Transitions
- Circle Tracking:
 - Update X/Y Coordinates, End of Frame
- Outputs: Center coordinates of markers
- Processing a frame would take about $(1280 \times 720 = 921600)$ clock cycles = 921600ns

Custom IP - Denoising

- Faced problems with joint detection due to black and white pixel noise
- Implemented several different denoising algorithms
 - Minimum width and height (white/colour pixel denoising)
 - Minimum gap length (black pixel denoising)
- Adjusting both b/w and rgb thresholds to minimize noise



Integration



Project is built off an HDMI tutorial by Digilent

2 AXI interconnects used, one is dedicated for memory read and writes, the other for processor control

AXI Uartlite and buttons used to control the system

Custom IP takes in frame through a VDMA, and stores outputs in slave registers. When done a frame, interrupt is sent to the MB processor

Timing

Clock tree:

MIG7 - Uses 200MHz sysclk

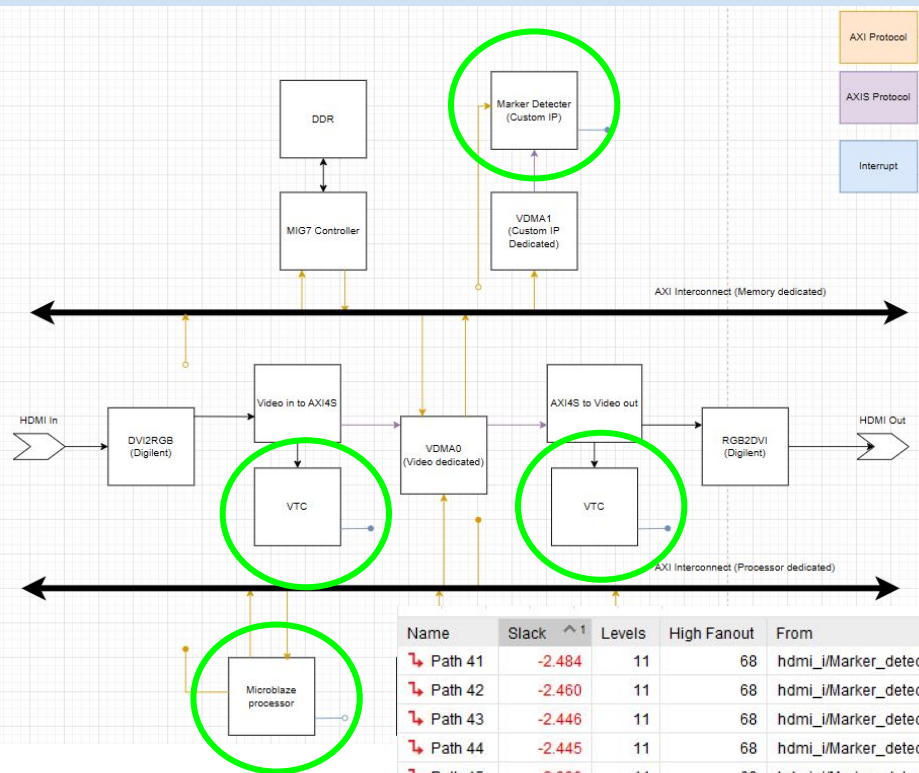
Entire system is run off a 100MHz clock generated by MIG

Processor timing:

Input and output frame timing of the HDMI system is dictated by VTCs through interrupts

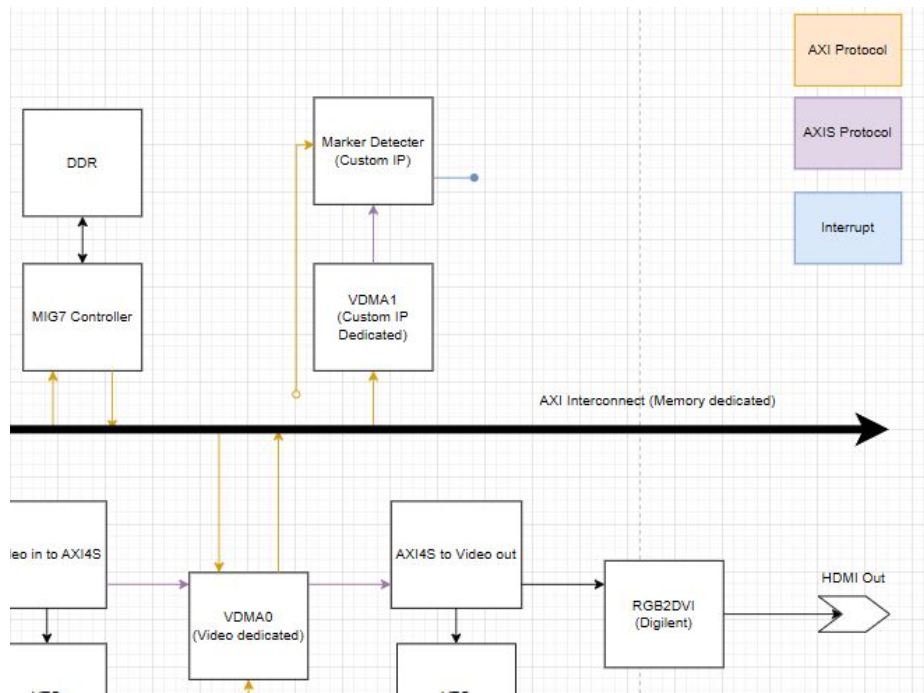
Independently, frame writing is dictated by custom IP's interrupt, uses a double buffer

Timing violation on custom IP (long term logic):



Name	Slack	Levels	High Fanout	From	To
Path 41	-2.484	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[5][2]/D
Path 42	-2.460	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[5][3]_replica/D
Path 43	-2.446	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[0][2]/D
Path 44	-2.445	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[5][1]/D
Path 45	-2.390	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[12][6]/D
Path 46	-2.362	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[8][7]/D
Path 47	-2.348	11	68	hdmi_i/Marker_detector_v4_0/inst/joint_detection/last_rows_reg[5][3]/C	hdmi_i/Marker_detector_v4_0/inst/joint_detection/x_mids_reg[4][10]/D

Dataflow



1280x720p Input video in RGB888 (2.7MB/frame)

Data storage

- 3 framebuffers stored in memory (8.1MB)

Dedicated AXI Interconnect for memory read/writes

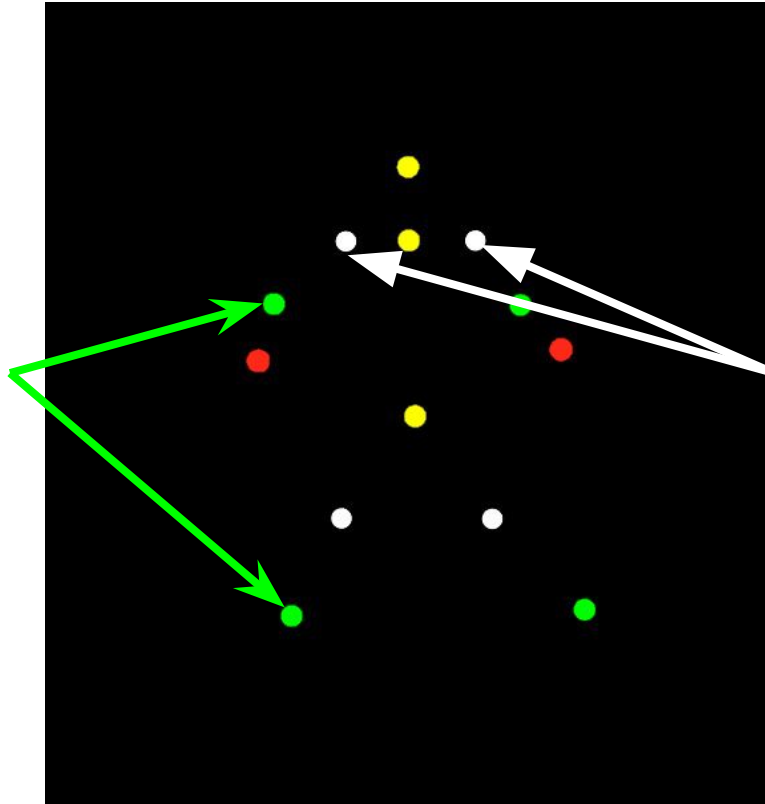
- VDMA0 does both read and writes to memory
- VDMA1 only does reads from memory
- Microblaze processor writes directly to framebuffers, but is negligible
- HDMI I/O FPS: 60Hz
- Custom IP FPS: ~24Hz (Variable)
- Total bus throughput: ~389MB/s

Software Flow - Video Mode

1. Choose a framebuffer that is not being used
2. Enable the Custom IP, and wait for a Valid Interrupt
3. Erase the previous frame's skeleton
4. Read the joint coordinates from the Custom IP
5. Draw the skeleton
 - a. For color mode, first classify each joint based on color and location
6. Draw GUI
7. Wait for previous frame to be displayed (VDMA MM2S/Vsync interrupt)
8. Display current frame

Software Flow - Video Mode Constraints

Upper row may
NEVER go below the
lower row



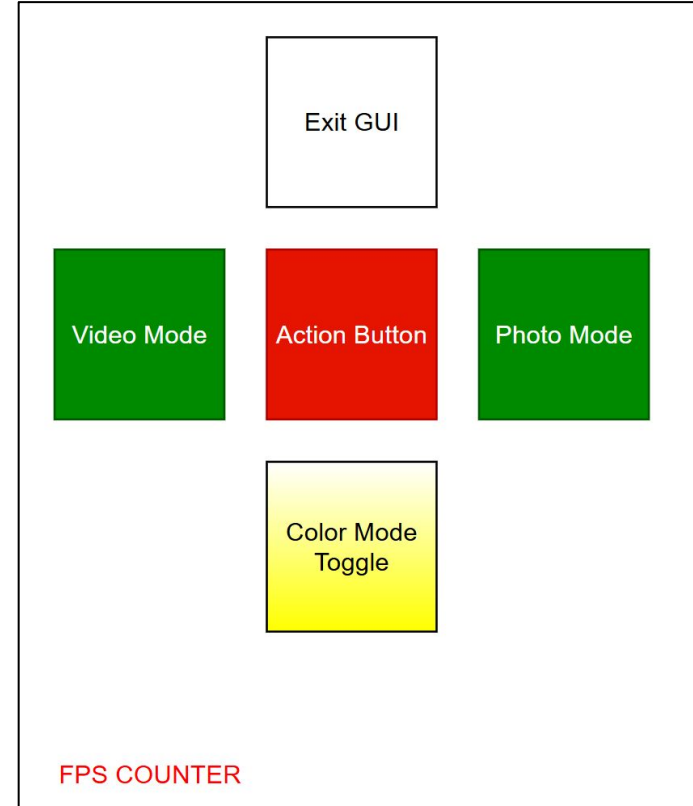
For each row, left
must NEVER pass
right

Software Flow - Video Mode

1. Choose a framebuffer that is not being used
2. Enable the Custom IP, and wait for a Valid Interrupt
3. Erase the previous frame's skeleton
4. Read the joint coordinates from the Custom IP
5. Draw the skeleton
 - a. For color mode, first classify each joint based on color and location
6. Draw GUI
7. Wait for previous frame to be displayed (VDMA MM2S/Vsync interrupt)
8. Display current frame

Software Flow - GUI

- Controlled by buttons on board
- Top - Exit GUI
- Left - Switch to Video Mode
- Middle - Action Button (behavior depends on mode)
- Right - Switch to Photo Mode
- Bottom - Color Mode Toggle (White or Color markers)
- FPS counter using AXI Timer/Counter

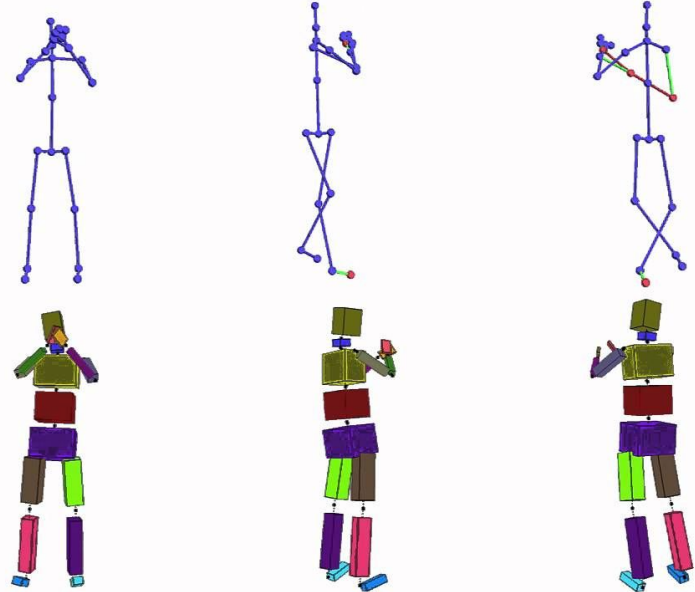


Complexity Points

Project Component	Complexity Points
Nexys Video	
HDMI input on Nexys Video	1
HDMI output on Nexys Video	1
Visualize meaningful results with a GUI	0.75
Custom IP	1.5
Software algorithm implemented on MicroBlaze	0.5
Total: 4.75 points	

Future Improvements

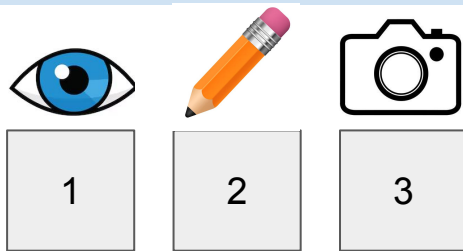
- Overlay a model onto the skeleton to achieve stretch goal
- Increase the number of detectable colours to remove constraints
- Improve denoising and colour detection
- Improve FPS
- 3D motion detection



Thank you!
Demo + Q&A

Software Flow - Video Mode Optimizations

- Double buffering: display one framebuffer, draw on one framebuffer, one framebuffer in reserve
- Optimized drawing algorithm:
 - Bresenham's Line Algorithm, only uses addition/subtraction/bit shifting
 - Manually shift the pixel address pointer using addition after finding the starting address
- Optimized erasing algorithm:
 - Store the location of the previous skeleton/markers and only draw over those locations (no need to use memset)



$$y = \frac{y_1 - y_0}{x_1 - x_0}(x - x_0) + y_0.$$

```
u32 addr = (y0 * stride) + (x0 * 3);  
addr += sx;  
addr += sy;
```